

1 Contest

2 Data structures

3 Graph

4 Strings

5 Bitwise

Contest (1)

template.cpp 28 lines

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp> // for PBDS
#include <ext/pb_ds/tree_policy.hpp> // for PBDS
using namespace __gnu_pbds; // for PBDS
using namespace std;
#define int long long int
const int MOD = 1000000007;

template<typename T>
using ordered_set = tree<T,null_type,less_equal<T>,rb_tree_tag,
    tree_order_statistics_node_update>;
// less_equal will make it multiset
// less makes it set
void solve() {

}
```

```
signed main() {
ios_base::sync_with_stdio(false);
cin.tie(NULL);
ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0); // orz
cin.exceptions(cin.failbit); // orz
    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

troubleshoot.txt 52 lines

Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?

1 Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
1 Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
3 Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
3 Rewrite your solution from the start or let a teammate do it.

5 Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Data structures (2)

SegmentTree.h
Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.
Usage: Node* tr = new Node(v, 0, sz(v));
Time: $\mathcal{O}(\log N)$.

```
struct Node {
    int sum;
    Node(int sum) : sum(sum) {}
    Node() : sum(0) {}
};
```

```
Node merge(Node a,Node b) {
    return Node(a.sum + b.sum);
}
```

```
void build(int idx,int l,int r,vector<int> &a) {
    if(l==r) {
        tree[idx] = {a[l]};
        return;
    }
    int mid = (l+r)/2;
    build(2*idx,l,mid,a);
    build(2*idx + 1,mid + 1,r,a);
    tree[idx] = merge(tree[2*idx],tree[(2*idx)+ 1]);
}
```

```
Node query(int idx,int tl,int th,int ql,int qh) {
    if(ql>th || qh<tl) {
        return Node();
    }
}
```

```
if(ql<=tl && th<=qh) {
    return tree[idx];
}
int mid = (tl+th)/2;
return merge(query(2*idx,tl,mid,ql,qh),query(2*idx + 1,mid
    + 1,th,ql,qh));
}
```

```
if(tl>pos || th<pos) {
    return;
}
if(tl==th) {
    tree[idx] = {val};
    return;
}
int mid = (tl+th)/2;
update(2*idx,tl,mid,pos,val);
update(2*idx + 1,mid + 1,th,pos,val);
tree[idx] = merge(tree[2*idx],tree[2*idx + 1]);
}
```

// you can change the struct to whatever you want
tree.resize(4*N);
build(1,1,N,a);
update(1,1,N,pos,x);
query(1,1,N,1,r).sum;

LazySegmentTree.h
Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.
Usage: Node* tr = new Node(v, 0, sz(v));
Time: $\mathcal{O}(\log N)$.

```
struct Node {
    int sum;
    int lazy_add;
    int lazy_set;
    Node(int sum,int lazy_add,int lazy_set) : sum(sum),lazy_add
        (lazy_add),lazy_set(lazy_set) {}
    Node() : sum(0),lazy_add(0),lazy_set(-1) {}
};
```

```
Node merge(Node a,Node b) {
    return Node(a.sum+b.sum,0,-1);
}
```

```
vector<Node> tree;
```

```
void apply_add(int idx,int l,int r,int val) {
    tree[idx].sum += (r-l+1)*val;
    if(tree[idx].lazy_set!=-1) {
        tree[idx].lazy_set += val;
    } else {
        tree[idx].lazy_add += val;
    }
}
```

```
void apply_set(int idx,int l,int r,int val) {
    tree[idx].sum = (r-l+1)*val;
    tree[idx].lazy_set = val;
    tree[idx].lazy_add = 0;
}
```

```
void build(int idx,int l,int r,vector<int> &a) {
    if(l==r) {
        tree[idx] = {a[l],0,-1};
        return;
    }
}
```

```
    }
    int mid = (l+r)/2;
    build(2*idx,l,mid,a);
    build(2*idx + 1,mid + 1,r,a);
    tree[idx] = merge(tree[2*idx],tree[(2*idx) + 1]);
}

void push(int idx,int l,int r) {
    if(l==r) {
        return;
    }
    int mid = (l+r)/2;
    if(tree[idx].lazy_set!=-1) {
        apply_set(2*idx,l,mid,tree[idx].lazy_set);
        apply_set(2*idx + 1,mid + 1,r,tree[idx].lazy_set);
        tree[idx].lazy_set = -1;
    }
    if(tree[idx].lazy_add!=0) {
        apply_add(2*idx,l,mid,tree[idx].lazy_add);
        apply_add(2*idx + 1,mid + 1,r,tree[idx].lazy_add);
        tree[idx].lazy_add = 0;
    }
}

Node query(int idx,int tl,int th,int ql,int qh) {
    if(ql<=tl && th<=qh) {
        return tree[idx];
    }
    if(ql>th || qh<tl) {
        return Node();
    }
    int mid = (tl+th)/2;
    push(idx,tl,th);
    return merge(query(2*idx,tl,mid,ql,qh),query(2*idx + 1,mid
        +1,th,ql,qh));
}

void update(int idx,int tl,int th,int ql,int qh,int val,int
    type) {
    if(ql>th || qh<tl) {
        return;
    }

    if(ql<=tl && th<=qh) {
        if(type==1) {
            // assign
            apply_set(idx,tl,th,val);
        } else if(type==2) {
            apply_add(idx,tl,th,val);
            // add
        }
        return;
    }
    int mid = (tl+th)/2;
    push(idx,tl,th);
    update(2*idx,tl,mid,ql,qh,val,type);
    update(2*idx + 1,mid + 1,th,ql,qh,val,type);
    tree[idx] = merge(tree[2*idx],tree[2*idx + 1]);
}

// // Usage :

// tree.resize(4*N);
// build(1,1,N,a);
// update(1,1,N,l,r,v,1);
// update(1,1,N,l,r,v,2);
// query(1,1,N,l,r).sum;
```

FenwickTree.h

Description: Computes partial sums $a[0] + a[1] + \dots + a[pos - 1]$, and updates single elements $a[i]$, taking the difference between the old and new value.

Time: Both operations are $\mathcal{O}(\log N)$.

f2c389, 18 lines

```
struct BIT {
    int size;
    vector<int> bit;

    BIT(int n) : size(n),bit(n+1){}

    void update(int x, int v) {
        x++;
        for (; x <= size; x += x & (-x)) { bit[x] += v; }
    }
    int query(int b) {
        b++;
        int result = 0;
        for (; b > 0; b -= b & (-b)) { result += bit[b]; }
        return result;
    }
};

FenwickTree2d.h
```

Description: Computes sums $a[i,j]$ for all $i < I, j < J$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).Time: $\mathcal{O}(\log^2 N)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

fd19a8, 24 lines

```

}
for(int i=LOG;i>=0;i--) {
    if(parent[a][i]!=parent[b][i]) {
        ans = min(ans,query[a][i]);
        ans = min(ans,query[b][i]);
        a = parent[a][i];
        b = parent[b][i]; // can add query info here
    }
}
ans = min(ans,query[a][0]);
ans = min(ans,query[b][0]);
return ans; // lca will be parent[a][0]
}

// Usage :
dfs(1,0,g,depth);
for(int i=1;i<=LOG;i++) {
    for(int j=1;j<=n;j++) {
        parent[j][i] = parent[parent[j][i-1]][i-1]; // here
        query also
        query[j][i] = min(query[j][i-1],query[parent[j][i-1]][i-1]);
    }
}
}
```

Graph (3)

ArticulationPoints.h

Description: Find Articulation Points
Time: $\mathcal{O}(V + E)$ 88f8ce, 26 lines

```

void dfs_ap(int u, int p, vector<bool>& is_ap) {
    tin[u] = low[u] = ++timer;
    int children = 0;

    for (int v : adj[u]) {
        if (v == p) continue;
        if (tin[v] != -1) {
            // Back-edge: v was visited before u
            low[u] = min(low[u], tin[v]);
        } else {
            // Tree-edge
            children++;
            dfs_ap(v, u, is_ap);
            low[u] = min(low[u], low[v]);

            // If not root and child v has no back-edge to u or
            // above
            if (p != -1 && low[v] >= tin[u]) {
                is_ap[u] = true;
            }
        }
    }
    // Special case: root is an AP if it has more than 1 child
    // in DFS tree
    if (p == -1 && children > 1) {
        is_ap[u] = true;
    }
}
}
```

Bridges.h

Description: Find Bridges
Time: $\mathcal{O}(V + E)$ ef0b16, 18 lines

```

void dfs_bridges(int u, int p, vector<pair<int, int>>& bridges)
{
    tin[u] = low[u] = ++timer;

    for (int v : adj[u]) {
```

```

        if (v == p) continue;
        if (tin[v] != -1) {
            low[u] = min(low[u], tin[v]);
        } else {
            dfs_bridges(v, u, bridges);
            low[u] = min(low[u], low[v]);

            // Bridge condition: no path from v to u or above
            if (low[v] > tin[u]) {
                bridges.push_back({u, v});
            }
        }
    }
}
}
```

SCC.h

Description: Converts Graph into a DAG.
Time: $\mathcal{O}(|V| + |E|)$ 15b328, 39 lines

```

void dfs(int u, vector<vector<int>> &g, vector<int> &visited,
    stack<int> &s) {
    visited[u] = 1;
    for(auto v : g[u]) {
        if(!visited[v]) dfs(v, g, visited, s);
    }
    s.push(u);
}

void dfs1(int u, vector<vector<int>> &g_dash, vector<int> &
    visited, int scc, vector<int> &res) {
    visited[u] = 1;
    res[u] = scc;
    for(auto v : g_dash[u]) {
        if(!visited[v]) dfs1(v, g_dash, visited, scc, res);
    }
}

vector<int> kosaraju(int n, vector<vector<int>> &g) {
    stack<int> s;
    vector<int> visited(n + 1, 0);
    for(int i = 1; i <= n; i++) {
        if(!visited[i]) dfs(i, g, visited, s);
    }
    vector<vector<int>> g_dash(n + 1);
    for(int i = 1; i <= n; i++) {
        for(auto v : g[i]) g_dash[v].push_back(i);
    }
    fill(visited.begin(), visited.end(), 0);
    vector<int> res(n + 1, 0);
    int scc_count = 0;

    while(!s.empty()) {
        int node = s.top();
        s.pop();
        if(!visited[node]) {
            scc_count++;
            dfs1(node, g_dash, visited, scc_count, res);
        }
    }
    return res;
}
}
```

HopcroftKarp.h

Description: Description: Fast bipartite matching algorithm. Graph *adj* should be a list of neighbors of the left partition, and *matchR* should be an array of size *M* initialized to 0. Returns the size of the matching in $\mathcal{O}(E\sqrt{V})$. *matchR[i]* will be the match for vertex *i* on the right side, or 0 if it is not matched.
Time: $\mathcal{O}(E\sqrt{V})$ 836465, 64 lines

```

vector<int> adj[MAXN];
int matchL[MAXN], matchR[MAXM], dist[MAXN];
int n, m; // n: size of U, m: size of V

bool bfs() {
    queue<int> q;
    for (int u = 1; u <= n; u++) {
        if (matchL[u] == 0) {
            dist[u] = 0;
            q.push(u);
        } else {
            dist[u] = INF;
        }
    }
    dist[0] = INF;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        if (dist[u] < dist[0]) {
            for (int v : adj[u]) {
                if (dist[matchR[v]] == INF) {
                    dist[matchR[v]] = dist[u] + 1;
                    q.push(matchR[v]);
                }
            }
        }
    }
    return dist[0] != INF;
}

bool dfs(int u) {
    if (u != 0) {
        for (int v : adj[u]) {
            if (dist[matchR[v]] == dist[u] + 1) {
                if (dfs(matchR[v])) {
                    matchR[v] = u;
                    matchL[u] = v;
                    return true;
                }
            }
        }
        dist[u] = INF;
        return false;
    }
    return true;
}

int hopcroft_karp() {
    int matching = 0;
    // Reset matches
    for (int i = 0; i <= n; i++) matchL[i] = 0;
    for (int i = 0; i <= m; i++) matchR[i] = 0;

    while (bfs()) {
        for (int u = 1; u <= n; u++) {
            if (matchL[u] == 0 && dfs(u)) {
                matching++;
            }
        }
    }
    return matching;
}
```

Strings (4)

doubleHasher.h

Description: Various self-explanatory methods for string hashing. Use on Codeforces, which lacks 64-bit support and where solutions can be hacked.

```
554336, 67 lines
struct hasher {
    int base;
    int mod;
    vector<int> hash;
    vector<int> basepow;
    vector<int> rHash;

    hasher() {}

    hasher(string s, int _base = 37, int _mod = 1e9 + 7) {
        base = _base;
        mod = _mod;
        int n = s.size();
        s = "#" + s + "#";

        hash.resize(n + 2);
        basepow.resize(n + 2);
        rHash.resize(n + 2);

        hash[0] = 0;
        basepow[0] = 1;

        for (int i = 1; i <= n; i++) {
            hash[i] = (hash[i - 1] * base + (s[i] - 'a' + 1)) %
                mod;
            basepow[i] = (basepow[i - 1] * base) % mod;
        }

        rHash[n + 1] = 0;
        for (int i = n; i >= 1; i--) {
            rHash[i] = (rHash[i + 1] * base + (s[i] - 'a' + 1))
                % mod;
        }
    }

    int getHash(int l, int r) {
        int res = hash[r];
        res -= (basepow[r - l + 1] * hash[l - 1]) % mod;
        res = (res % mod + mod) % mod;
        return res;
    }

    int getRevHash(int l, int r) {
        int res = rHash[l];
        res -= (basepow[r - l + 1] * rHash[r + 1]) % mod;
        res = (res % mod + mod) % mod;
        return res;
    }

    bool isPalindrome(int l, int r) {
        return getHash(l, r) == getRevHash(l, r);
    }
};

struct doubleHasher{
    hasher hash1,hash2;

    doubleHasher() {}

    doubleHasher(string s) {
        hash1 = hasher(s, 53, 1000000007);
        hash2 = hasher(s, 53, 1000000009);
    }
}
```

SAM KMP

```
int getHash(int l,int r) {
    return (1LL<<32)*hash1.getHash(l,r) + hash2.getHash(l,r
    );
}
};
```

SAM.h

Description: Suffix Automaton

```
e92027, 54 lines
struct state{
    int len,link;
    int next[26];
    // bool is_clone; // occurence counting
    // int first_pos; // finding first position
};

const int MAXN = 1e5;
state st[2*MAXN];
int sz,last;

void sa_init() {
    st[0].len = 0;
    st[0].link = -1;
    memset(st[0].next,-1,sizeof(st[0].next));
    sz = 1;
    last = 0;
}

void sa_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    // st[cur].is_clone = false;
    // st[cur].first_pos = st[cur].len - 1;
    memset(st[cur].next,-1,sizeof(st[cur].next));

    int p = last;
    int code = c - 'a';
    while(p!=-1 && st[p].next[code]==-1) {
        st[p].next[code] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[code];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            st[clone].link = st[q].link;
            memcpy(st[clone].next, st[q].next, sizeof(st[q].
                next));
            // st[clone].is_clone = true;
            // st[clone].first_pos = st[q].first_pos;
            while (p != -1 && st[p].next[code] == q) {
                st[p].next[code] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
```

Number of Distinct Substrings: Each state u represents substrings of lengths $(len(link(u)), len(u)]$, calculated as:

$$\sum_{u=1}^{sz-1} (len(u) - len(link(u)))$$

Total Length of Distinct Substrings: Arithmetic sum per state:

$$\sum_{u=1}^{sz-1} \frac{(len(u) - len(link(u)))(len(u) + len(link(u)) + 1)}{2}$$

Occurrence Count: Initialize $cnt[u] = 1$ for non-clones, 0 for clones; sort states by decreasing len ; propagate $cnt[link(u)] += cnt[u]$.

First Occurrence Position: Store $first_pos$: new state $= len(cur) - 1$, clone $= first_pos(q)$; query answer $first_pos(u) - |P| + 1$.

All Occurrence Positions: Store inverse links $adj[link(u)].push_back(u)$; DFS/BFS from state u on inverse link tree (ignore clones).

Longest Common Substring (2 Strings): Build SAM of S , stream T keeping node v and length l : if $next[v][c]$ exists, $v = next[v][c]$, $l + +$; else jump $v \rightarrow link(v)$, $l = len(v)$ until valid. Max l is LCS.

LCS of K Strings: Build SAM of $T = S_1\#_1S_2\#_2 \dots S_K\#_K$; propagate per-state reachability (bitmask/count); answer is max $len(u)$ reachable by all K .

Shortest Non-Appearing String: BFS/DP on SAM graph (not links), $dp[u] = 1 + \min_{(u,v) \in edges} dp[v]$; shortest path from root to state with missing transition.

Smallest Cyclic Shift: Build SAM for $S + S$, from root greedily take smallest lex edge for exactly $|S|$ steps.

K-th Lexicographical Substring: Compute DAG paths $paths[u] = 1 + \sum_{(u,v) \in edges} paths[v]$, greedily traverse from root subtracting subtree sizes until K hits.

KMP.h

Description: $pi[x]$ computes the length of the longest prefix of s that ends at x , other than $s[0...x]$ itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

```
6cca8e, 22 lines
vi pi_function(string s) {
    int n = ssize(s);
    vi pi(n, 0);
    for (int i = 1, j = 0; i < n; i++) {
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) pi[i] = ++j;
    }
    return pi;
}

// Find occurrences of pat in text
int kmp(string text, string pat) {
    auto pi = pi_function(pat);
    int res = 0;
    for (int i = 0, j = 0; i < ssize(text); i++) {
        while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
        if (text[i] == pat[j]) ++j;
        if (j == ssize(pat))
    }
```

```
        res++; // occurrence starting at text[i-pat.size()
              +1] found
    }
    return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

a0719e, 20 lines

```
vi z_function(string s) {
    int n = ssize(s);
    vi z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[i + z[i]] == s[z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    // z[0] = n; // add if you need z[0]=n
    return z;
}

int z_algorithm(string text, string pat) { // Find occurrences
    of pat in text
    auto z = z_function(pat + '\0' + text);
    int res = 0;
    for (int i = 0; i < ssize(text) - ssize(pat) + 1; i++)
        if (z[i + ssize(pat) + 1] == ssize(pat))
            res++; // occurrence starting at text[i] found
    return res;
}
```

Trie.h

Description: Search, Delete, Add Words
Time: $\mathcal{O}(\text{AlphaBet})$

053287, 59 lines

```
struct trieNode {
    trieNode* child[26];
    int freq; // Number of words sharing this prefix
    int endCount; // Number of words ending exactly here

    trieNode() {
        for (int i = 0; i < 26; i++) child[i] = nullptr;
        freq = 0;
        endCount = 0;
    }
};

trieNode* root = new trieNode();

// Adds a string to the Trie
void add(string word) {
    trieNode* cur = root;
    cur->freq++;
    for (char c : word) {
        int idx = c - 'a';
        if (!cur->child[idx]) {
            cur->child[idx] = new trieNode();
        }
        cur = cur->child[idx];
        cur->freq++;
    }
    cur->endCount++;
}

// Counts occurrences of a specific word
int countWord(string word) {
    trieNode* cur = root;
```

Zfunc Trie Xor Basis Max XOR AOS

```
for (char c : word) {
    int idx = c - 'a';
    if (!cur->child[idx] || cur->child[idx]->freq == 0)
        return 0;
    cur = cur->child[idx];
}
return cur->endCount;
}

// Checks if a word exists (at least once)
bool search(string word) {
    return countWord(word) > 0;
}

// Deletes one instance of a string
void del(string word) {
    if (countWord(word) == 0) return; // Word doesn't exist,
    nothing to delete

    trieNode* cur = root;
    cur->freq--;
    for (char c : word) {
        int idx = c - 'a';
        trieNode* next = cur->child[idx];
        next->freq--;
        cur = next;
    }
    cur->endCount--;
}
```

Bitwise (5)

Xor Basis.h

Description: XOR Basis for Max Subset Xor and Number of Subsets
Time: Both operations are $\mathcal{O}(\log N)$.

f1749f, 34 lines

```
void solve() {
    int n;
    cin>>n;
    vector<int> a(n);
    for(int i=0;i<n;i++) {
        cin>>a[i];
    }
    sort(a.begin(),a.end());
    int index = 0;
    for(int i=32;i>=0;i--) {
        int temp = -1;
        for(int j=index;j<n;j++) {
            if(a[j]&(1LL<<i)) {
                temp = j;
                break;
            }
        }
        if(temp==-1) {
            continue;
        }
        swap(a[index],a[temp]);
        for(int j=0;j<n;j++) {
            if(a[j]&(1LL<<i) && j!=index) {
                a[j] = a[j]^a[index];
            }
        }
        index++;
    }
    int ans = 0;
    for(int i=0;i<n;i++) {
        ans = max(ans,ans^a[i]);
    }
}
```

```
        cout<<ans<<endl;
    }
}

Max XOR.h
Description: Using Tries to extract max subarray XOR
Time: Both operations are  $\mathcal{O}(\log N)$ .
857b00, 35 lines

struct trie {
    int val;
    trie* arr[2];
    trie() {
        arr[0] = NULL;
        arr[1] = NULL;
    }
};

void insert(trie *root,int n) {
    trie* temp = root;
    for(int i=32;i>=0;i--) {
        bool x = n&(1LL<<i);
        if(temp->arr[x]!=NULL) {
            temp = temp->arr[x];
        } else {
            temp->arr[x] = new trie();
            temp = temp->arr[x];
        }
    }
    temp->val = n;
}

int query(trie *root,int n) {
    trie* temp = root;
    for(int i=32;i>=0;i--) {
        bool x = n&(1LL<<i);
        if(temp->arr[!x]!=NULL) {
            temp = temp->arr[!x];
        } else if(temp->arr[x]!=NULL) {
            temp = temp->arr[x];
        }
    }
    return n^temp->val;
}

//Usage :
//trie *root = new trie();

AOS.h
Description: Addition and Subtraction Over Subsets
Time: Both operations are  $\mathcal{O}(\log N)$ .
175b47, 55 lines

// MLOG represents number of bits, MAXN is total masks (2^MLOG)
const lli MLOG = 20;
const lli MAXN = (1LL << MLOG);

// Forward 1: Subset Sum (SOS DP). Adds value of each element
to all its supersets.
void forward1(lli dp[]) {
    for (int bit = 0; bit < MLOG; bit++) {
        for (int i = 0; i < MAXN; i++) {
            // Check if the specific bit is set in the current
            mask i
            if (i & (1 << bit)) {
                // dp[mask] = sum of dp[submask], moving from
                smaller to larger masks
                dp[i] += dp[i ^ (1 << bit)];
            }
        }
    }
}

// Backward 1: Inverse Subset Sum. Subtracts submask values to
restore original array.
```

```
void backward1(lli dp[]) {
    for (int bit = 0; bit < MLOG; bit++) {
        // Iterate backward to reverse the prefix sum
        // accumulation correctly
        for (int i = MAXN - 1; i >= 0; i--) {
            if (i & (1 << bit)) {
                // Subtract the contribution of the submask (
                // where bit is 0)
                dp[i] -= dp[i ^ (1 << bit)];
            }
        }
    }
}

// Forward 2: Superset Sum. Adds value of each element to all
// its subsets.
void forward2(lli dp[]) {
    for (int bit = 0; bit < MLOG; bit++) {
        for (int i = MAXN - 1; i >= 0; i--) {
            // Check if the specific bit is set in the current
            // mask i
            if (i & (1 << bit)) {
                // Add the value of the supermask to the
                // submask (flipping bit from 1 to 0)
                dp[i ^ (1 << bit)] += dp[i];
            }
        }
    }
}

// Backward 2: Inverse Superset Sum. The inverse operation of
// forward2.
void backward2(lli dp[]) {
    for (int bit = 0; bit < MLOG; bit++) {
        // Iterate forward to reverse the superset accumulation
        // correctly
        for (int i = 0; i < MAXN; i++) {
            if (i & (1 << bit)) {
                // Remove the contribution of the supermask
                // from the submask
                dp[i ^ (1 << bit)] -= dp[i];
            }
        }
    }
}
```